CITS 3002
Computer Networks

Lab 5: Routing Protocols

Note: This lab runs for multiple weeks, so don't panic if you don't finish in the first week.

Worth: 2% of the unit

Please note: ask your lab demonstrator about any questions you have regarding previous labs. It will be beneficial for you to have a good foundation early on in the semester, and we are here to help!

In this lab, we will cover routing protocols, expanding the flooding we did in lab 4. The length of the lab has taken into account that the project is running in parallel.

1. Tasks

TASK 1. FLOODING - REVISITED

Create a new directory for this labsheet, and take a copy of and expand the ZIP archive file. Before examining any source code files, read the two files `1st.README` and `2nd.README`.

Compare the 3 flooding protocols by running the provided scripts:

```
python runall.py
python plotstats.py stats_FLOODING1.csv stats_FLOODING2.csv stats_FLOODING3.csv
```

Observe how the total number of messages delivered does not vary significantly between the three protocols.

Try plotting some other statistics, such as Efficiency (AL/PL) or Bytes Received (Physical), to see if there is any notable improvement across the three protocols. Why do the improved protocols not result in a higher rate of message delivery? Under what circumstances would they improve the rate of message delivery?

All 3 flooding implementations share an implementation of a simple Network Layer routing table, in `nltable.py`.

Examine the implementation of `down_to_datalink` and `up_to_datalink` in the flooding protocols (all three protocols use the same implementation of the datalink layer). It avoids any frame loss and corruption at the Physical layer (it cheats!) by calling `write_physical_reliable()` instead of `write_physical()`. Because "nothing can go wrong", we don't need to manage any sequence numbers or buffers of frames in this layer, and our Frame object can consist of just its payload (the NL's packets). Of course, this is rather unrealistic. Consider what effect un-reliable links would have on this type of approach. What changes might you need to replace this implementation with a `stopandwait` protocol?

All 3 flooding protocols limit the maximum distance travelled by any single message, through the use of the constant `MAXHOPS`. This is set to 4 `for` for our topologies, but for a larger map, such a limitation will actually prevent distant nodes from communicating at all.

Take a copy of `FLOODING3`, add a few more nodes and links to make the distance from, say, Perth to HongKong, 6 hops. Now observe how the `flooding3` protocol will fail to deliver messages.

Can you devise a modification to the `flooding3` protocol to support topologies with lengthy paths?

TASK 2. DISTANCE-VECTOR ROUTING

When considering network routing approaches there is a trade-off between centralised and de-centralised approaches. Flooding is arguably the *most* de-centralised approach we can concoct; nodes do not store any routing information. Distance-vector routing (as mentioned in the lectures) is a decentralisation of the Bellman-Ford algorithm for all-pairs shortest-paths.

The aim of this task is to understand the algorithm fully. Thinking about algorithms in a decentralised manner is often significantly harder than a centralised approach. While initially successful, distance-vector routing comes across some interesting problems which we would like you to investigate. Think carefully about how Distance-Vector behaves when links break.

The Distance-Vector Routing protocol provides a simple mechanism for routing messages:

Periodically, say every 30 seconds, nodes exchange information with their direct neighbours about how 'far away' are other nodes in the network. Providing nodes and links don't crash, and the number of nodes in the network is fixed, the Distance Vector Routing protocol enables convergence of routing information, but can also adapt to varying per-node and per-link conditions.

Implement the Distance Vector Routing protocol in the simulator, using the number of hops as a routing metric. Initially, do not generate or deliver any messages (no node should call `enable_application()`). Instead, just focus on generating, transmitting, receiving, and merging the routing update packets. You can start with the basic file `distancevector.py`, which already provides one possible implementation for storing the local routing table, as well as methods to pack the table for transmission over the network, and unpack transmitted tables from the network. When a node's routing table is updated by a message from the network, print the complete contents of the updated routing table for that node. Over time, the detail in the table should 'grow' until it stabilises to reflect all connection information in the simulation.

2. Quiz on LMS

To gain 2% allocated mark for this lab, please ensure that you solve lab questions on LMS by the deadline.